

Baraza Banking Platform

Cloud-Native Core Banking Platform built with Spring Boot, Kubernetes, Helm, Terraform, GitOps, and modern Platform Engineering practices.

Table of Contents

- [1. Overview](#)
- [2. Project Goals](#)
- [3. Platform Architecture](#)
- [4. Core Banking Services](#)
- [5. Technology Stack](#)
- [6. Repository Structure](#)
- [7. Application Architecture](#)
- [8. Containerization](#)
- [9. Kubernetes Platform](#)
- [10. Helm](#)
- [11. Environment Configuration](#)
- [12. Database](#)
- [13. Networking](#)
- [14. Health Checks and Reliability](#)
- [15. Horizontal Pod Autoscaling](#)
- [16. Infrastructure as Code](#)
- [17. CI/CD](#)
- [18. GitOps and Argo CD](#)
- [19. Monitoring and Observability](#)
- [20. Security](#)
- [21. Local Development Environment](#)
- [22. Running the Platform](#)
- [23. Building the Applications](#)
- [24. Building Docker Images](#)
- [25. Deploying to Kubernetes](#)
- [26. Helm Deployment](#)
- [27. GitOps Deployment](#)
- [28. Verification](#)
- [29. Troubleshooting](#)
- [30. Production Architecture](#)
- [31. Banking and Regulatory Considerations](#)
- [32. Disaster Recovery](#)
- [33. Development Workflow](#)
- [34. Platform Engineering Principles](#)
- [35. Current Project Status](#)
- [36. Roadmap](#)
- [37. Future Improvements](#)

- [38. Documentation](#)
- [39. Contributing](#)
- [40. License](#)

1. Overview

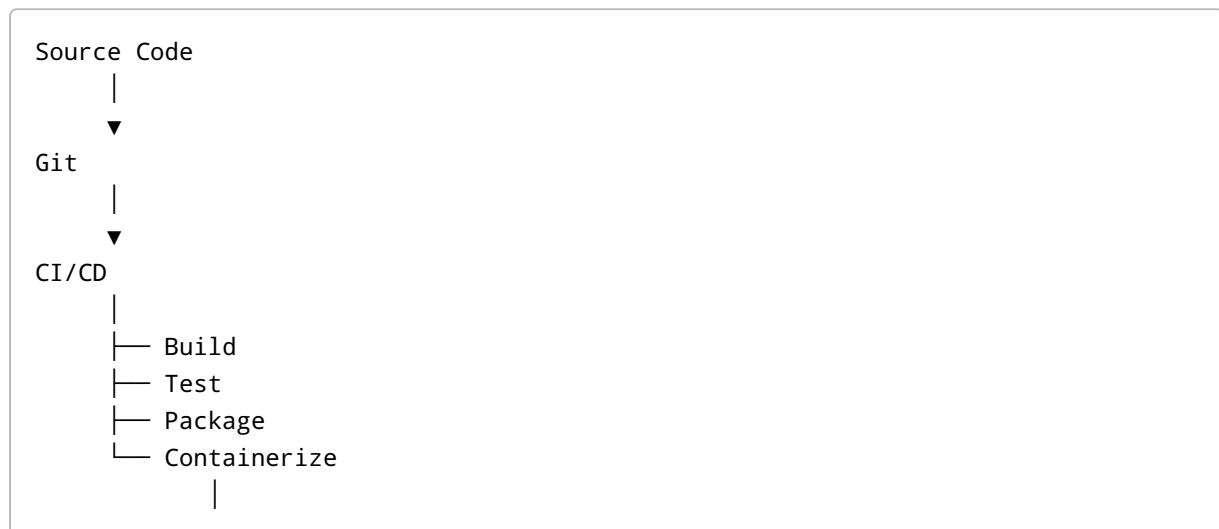
Baraza Banking Platform is a cloud-native banking platform designed to demonstrate how a modern banking workload can be developed, containerized, deployed, operated, and scaled using modern DevOps and Platform Engineering practices.

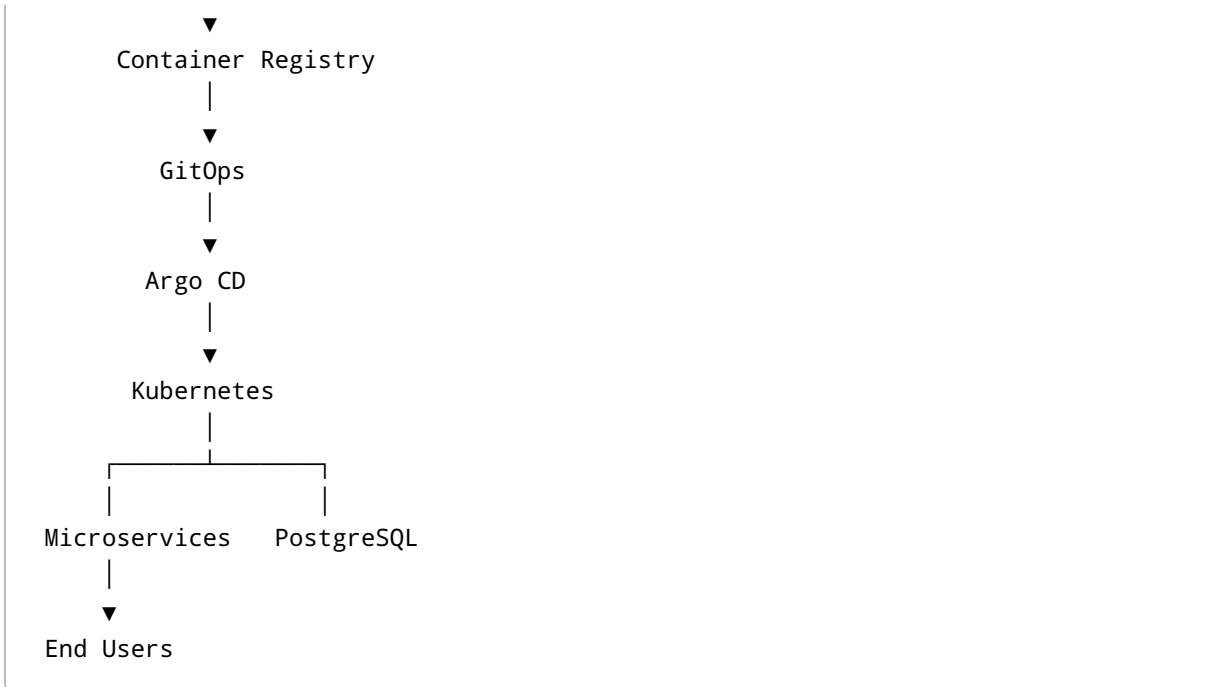
The platform follows a microservices architecture and is designed around:

- Spring Boot
- Java
- PostgreSQL
- Docker
- Kubernetes
- Helm
- Terraform
- GitOps
- Argo CD
- CI/CD
- Monitoring and observability
- Infrastructure as Code
- Security and reliability practices

The project is intentionally designed not only as an application development project, but also as a **Platform Engineering learning and demonstration environment**.

The goal is to demonstrate the complete lifecycle:





2. Project Goals

The primary goals of the Baraza Banking Platform are to demonstrate:

Application Engineering

- Microservice architecture
- REST APIs
- Spring Boot services
- Java 21
- Database integration
- Service-to-service communication
- Application health management

Containerization

- Docker image creation
- Multi-stage/container build practices
- Minimal runtime images
- Non-root containers
- Image versioning
- Container security

Kubernetes

- Deployments

- Services
- Namespaces
- ConfigMaps
- Secrets
- Ingress
- Probes
- Resource requests and limits
- Horizontal Pod Autoscaling
- Rolling deployments

Platform Engineering

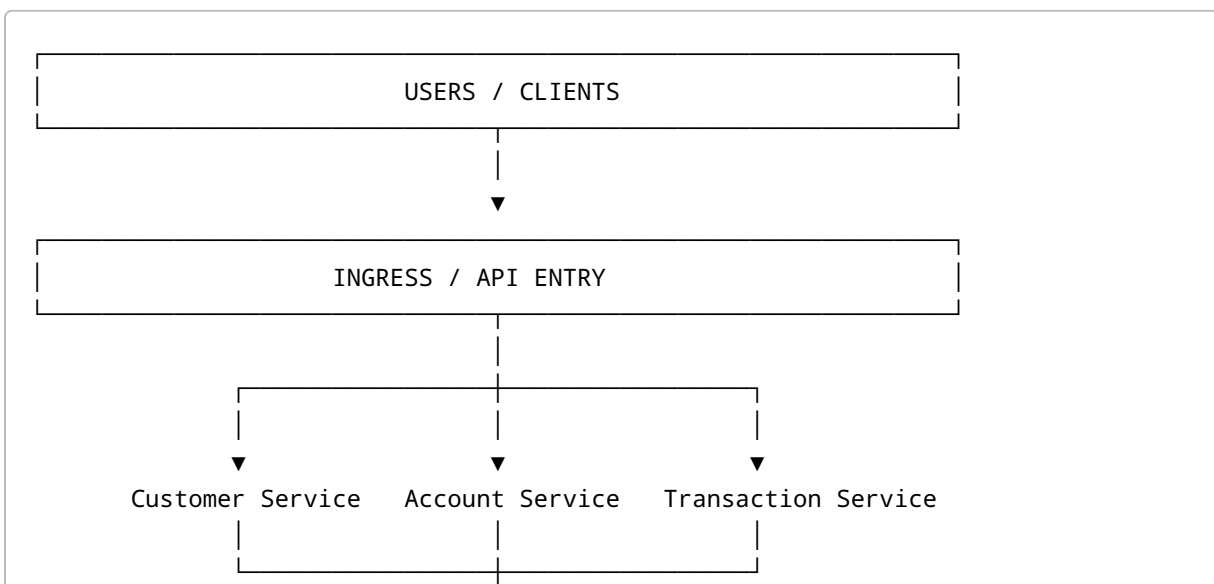
- Infrastructure as Code
- Environment management
- Reusable Helm charts
- GitOps
- Automated deployments
- Observability
- Reliability
- Operational runbooks

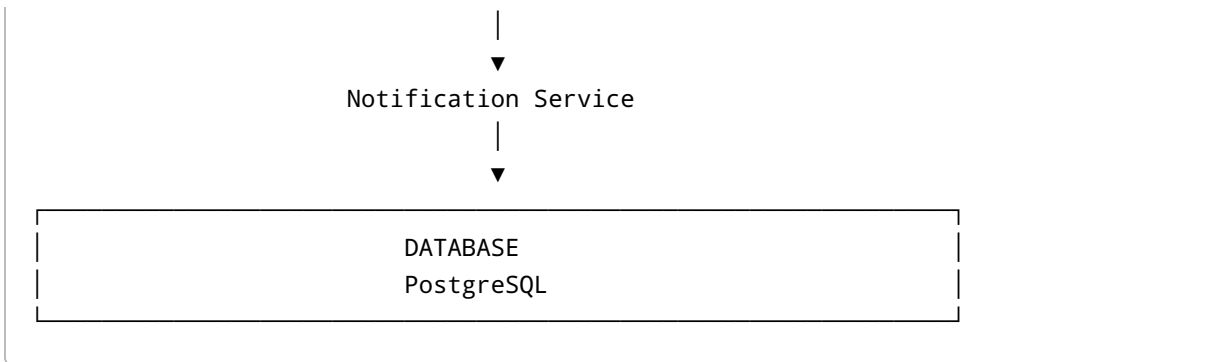
Cloud Engineering

The platform is designed so that the Kubernetes environment can eventually be deployed onto a cloud provider such as AWS.

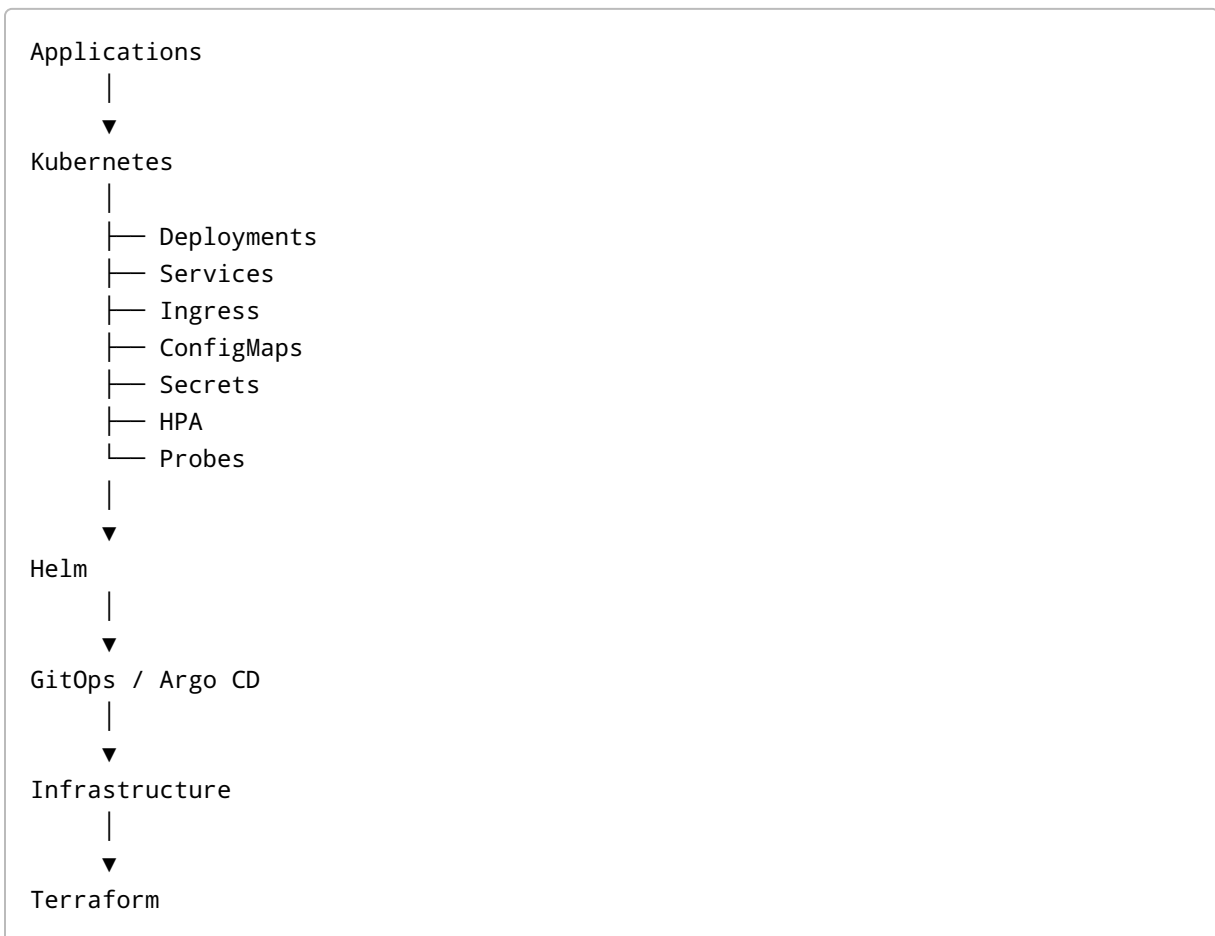
3. Platform Architecture

The platform follows a layered architecture.





The infrastructure layer sits underneath the application:



4. Core Banking Services

The platform is organized into independent services.

4.1 Customer Service

Responsible for customer-related functionality.

Typical responsibilities include:

- Customer registration
 - Customer information
 - Customer lookup
 - Customer lifecycle management
-

4.2 Account Service

Responsible for bank account functionality.

Typical responsibilities include:

- Account creation
 - Account lookup
 - Account status
 - Account ownership
 - Account balances
-

4.3 Transaction Service

Responsible for financial transaction processing.

Typical responsibilities include:

- Deposits
- Withdrawals
- Transfers
- Transaction validation
- Transaction history
- Transaction state management

Financial transaction processing should be treated as a critical workload requiring strong consistency, auditability, idempotency, and security.

4.4 Notification Service

Responsible for customer notifications.

Potential notification channels include:

- Email
- SMS
- Push notifications

The service is separated from transaction processing so that notification workloads do not unnecessarily block core banking operations.

5. Technology Stack

Category	Technology
Language	Java 21
Framework	Spring Boot
Build Tool	Maven
Database	PostgreSQL
Container Runtime	Docker
Orchestration	Kubernetes
Local Kubernetes	Minikube
Packaging	Helm
Infrastructure as Code	Terraform
GitOps	Argo CD
CI/CD	GitHub-based pipeline
Monitoring	Prometheus / Metrics Server / monitoring stack
Source Control	Git / GitHub
Operating Environment	Linux containers
Local Development	Windows + Docker Desktop + Minikube

The exact versions and deployed components should be kept synchronized with the repository as the platform evolves.

6. Repository Structure

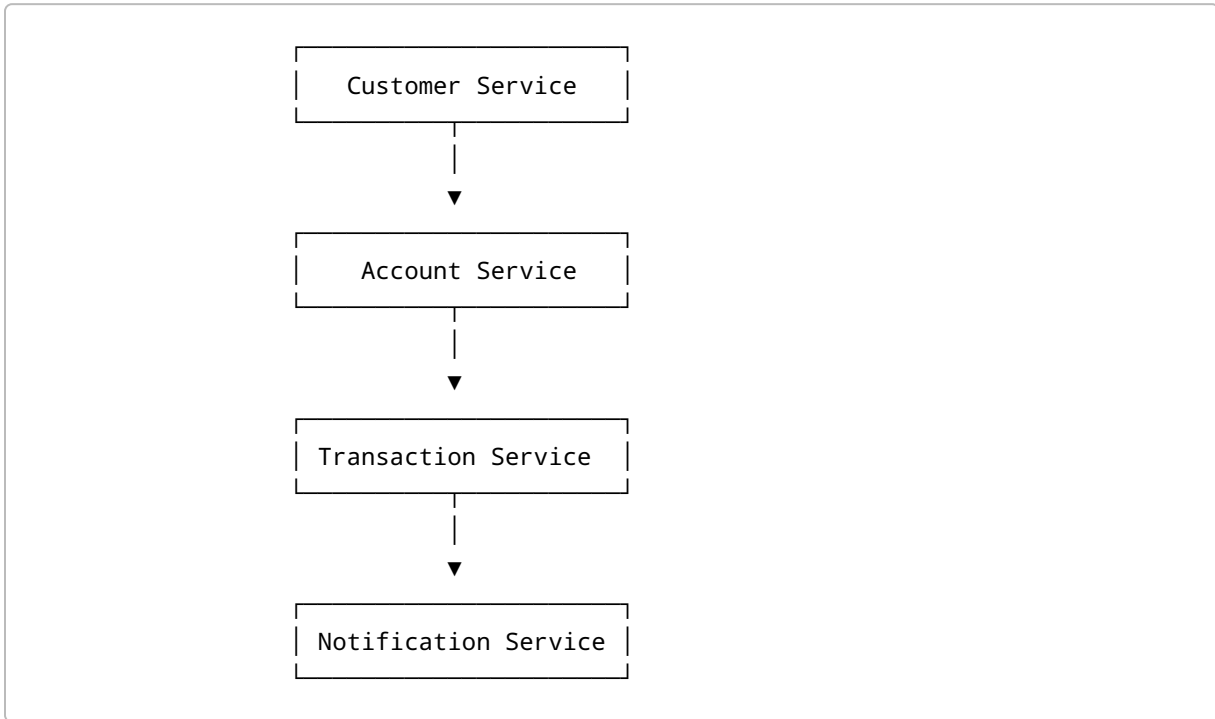
The repository is organized around applications and platform infrastructure.

```
baraza-banking-platform1/
├── applications/
│   ├── account-service/
│   │   ├── src/
│   │   ├── pom.xml
│   │   └── Dockerfile
│   ├── customer-service/
│   │   ├── src/
│   │   ├── pom.xml
│   │   └── Dockerfile
│   ├── transaction-service/
│   │   ├── src/
│   │   ├── pom.xml
│   │   └── Dockerfile
│   └── notification-service/
│       ├── src/
│       ├── pom.xml
│       └── Dockerfile
├── infrastructure/
│   ├── kubernetes/
│   ├── helm/
│   ├── terraform/
│   └── monitoring/
├── docs/
├── .github/
│   └── workflows/
├── README.md
└── ...
```

The exact repository structure may change as the platform develops.

7. Application Architecture

Each business capability is implemented as an independently deployable service.



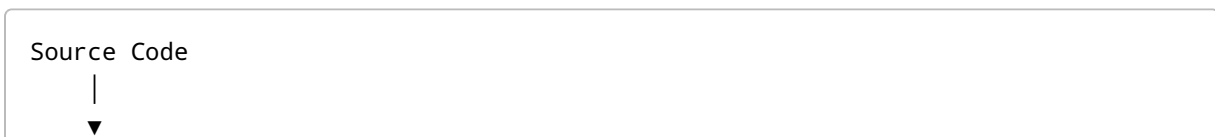
The microservice model provides:

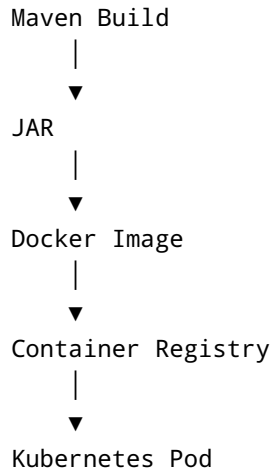
- Independent deployment
- Independent scaling
- Service isolation
- Clear ownership boundaries
- Failure isolation
- Technology flexibility
- Easier evolution of individual capabilities

8. Containerization

Every application is packaged as a container image.

A typical runtime architecture is:





The services use a Java runtime image such as:

```
eclipse-temurin:21-jre
```

Containers should run as a non-root user wherever possible.

Example:

```
USER 1001
```

This reduces the impact of a container compromise.

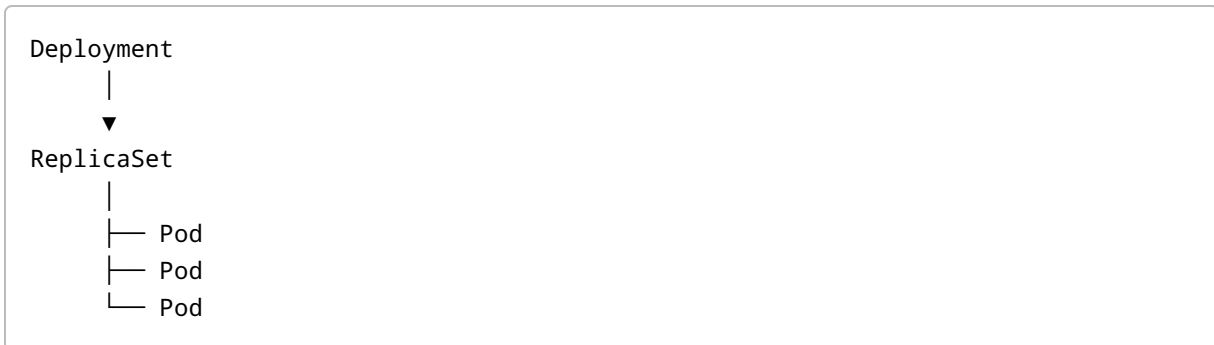
9. Kubernetes Platform

Kubernetes is responsible for orchestrating the application workloads.

The platform uses Kubernetes resources including:

- Namespace
- Deployment
- ReplicaSet
- Pod
- Service
- Ingress
- ConfigMap
- Secret
- HorizontalPodAutoscaler

Example deployment relationship:



The Deployment manages the desired state of the application.

10. Helm

Helm is used to package and template Kubernetes resources.

Instead of maintaining separate hard-coded manifests for every environment, configuration is parameterized.

For example:

```
replicaCount: 2
```

The deployment template can reference:

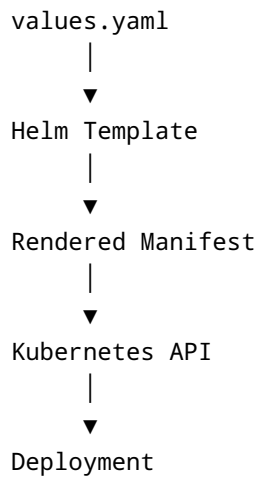
```
spec:
  replicas: {{ .Values.replicaCount }}
```

Helm renders the template into a Kubernetes manifest:

```
spec:
  replicas: 2
```

The resulting manifest is then submitted to Kubernetes.

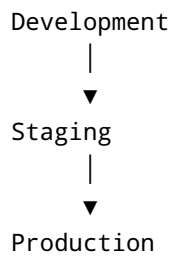
The flow is:



11. Environment Configuration

The platform supports environment-specific configuration.

Typical environments include:



Configuration can include:

- Replica count
- Image tag
- Service configuration
- Resource requests
- Resource limits
- Ingress hostname
- Probe configuration
- Environment variables

Example:

```
values-dev.yaml
values-prod.yaml
```

The application code remains the same while environment-specific configuration changes through deployment configuration.

12. Database

PostgreSQL is used as the relational database.

The database layer is responsible for persistent application state.

The architecture separates:

```
graph TD
    A[Application Layer] --> B[Persistence Layer]
    B --> C[PostgreSQL]
```

Database credentials should never be hard-coded into application source code.

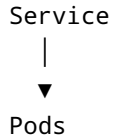
Kubernetes Secrets or a dedicated external secrets-management solution should be used for sensitive credentials.

13. Networking

Kubernetes Services provide stable networking for workloads.

A simplified networking model is:

```
graph TD
    A[Client] --> B[Ingress]
    B --> C[ ]
```



Pods are ephemeral and can be replaced at any time.

The Kubernetes Service provides a stable abstraction over those Pods.

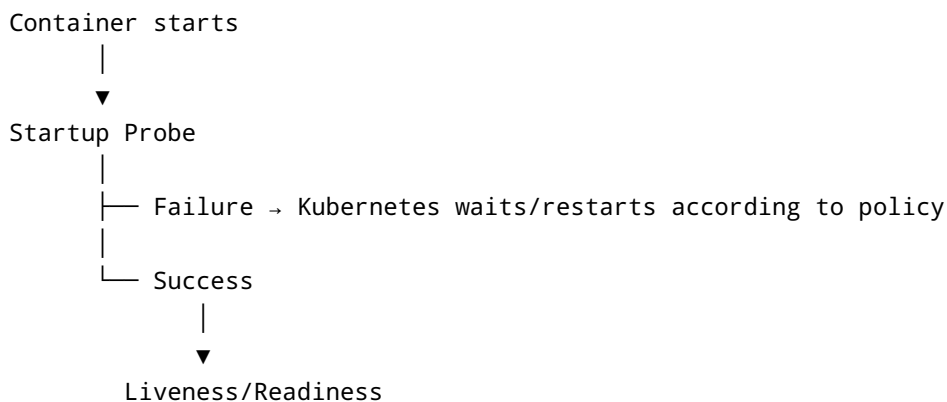
14. Health Checks and Reliability

Applications should expose health information that Kubernetes can use to determine application state.

The platform uses Kubernetes probes including:

Startup Probe

Determines whether the application has successfully started.



Liveness Probe

Determines whether the application is still alive.

Readiness Probe

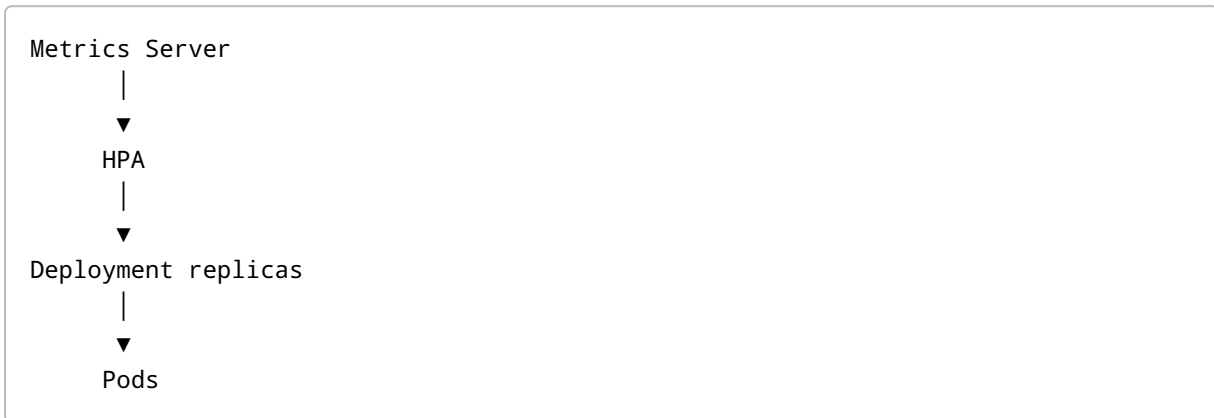
Determines whether the application is ready to receive traffic.

This prevents traffic from being sent to an application that has started its process but is not yet ready.

15. Horizontal Pod Autoscaling

Horizontal Pod Autoscaling allows Kubernetes to adjust the number of Pods based on resource utilization.

The relationship is:



For example:

```
MINPODS = 2
MAXPODS = 5
CPU target = 70%
Memory target = 80%
```

If workload increases and the configured utilization thresholds are exceeded, the HPA can increase the number of replicas.

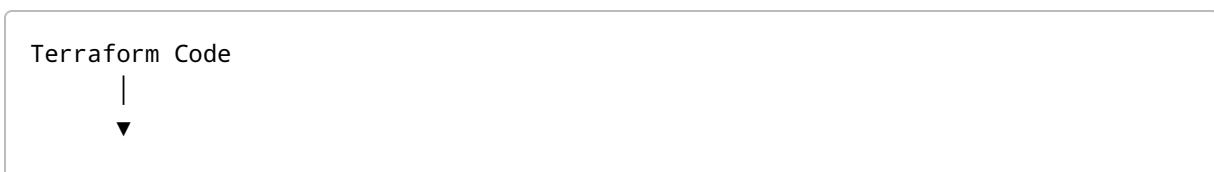
If demand decreases, Kubernetes can reduce replicas subject to the HPA policy.

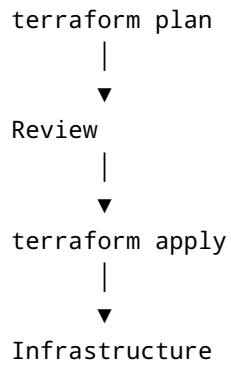
The Deployment's desired replica count and HPA behavior should therefore be understood together.

16. Infrastructure as Code

Terraform is used to define infrastructure declaratively.

The intended infrastructure lifecycle is:





Infrastructure as Code provides:

- Repeatability
- Version control
- Reviewability
- Consistency
- Automation
- Disaster recovery capability

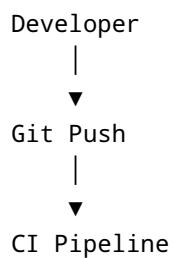
The Terraform layer can eventually manage cloud infrastructure including:

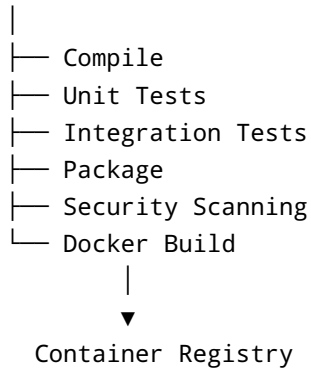
- VPC
- Subnets
- Security Groups
- IAM
- Kubernetes cluster
- Load balancers
- Databases
- Monitoring infrastructure

17. CI/CD

The CI/CD pipeline automates application delivery.

A typical pipeline is:



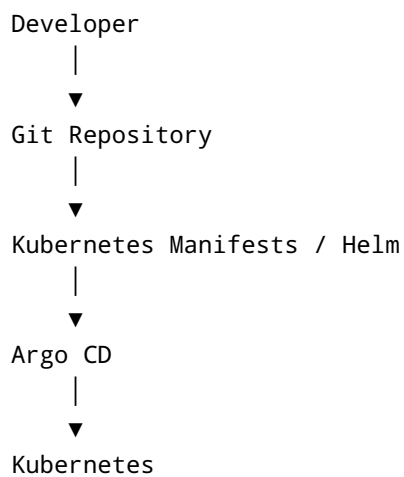


CI should prevent defective code from reaching deployment environments.

18. GitOps and Argo CD

GitOps makes Git the source of truth for the desired deployment state.

The intended model is:



Argo CD continuously compares:

Desired State
VS
Actual State

If the cluster differs from the desired state, Argo CD can synchronize the environment according to the configured policy.

This provides:

- Declarative deployments
- Auditability
- Version history
- Rollback capability
- Drift detection
- Reproducibility

19. Monitoring and Observability

The platform is designed to provide visibility into application and infrastructure health.

Observability should cover:

Metrics

Examples:

- CPU utilization
- Memory utilization
- Request rates
- Error rates
- Latency
- Pod count
- HPA activity

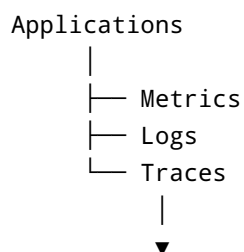
Logs

Application and Kubernetes logs should be centrally accessible.

Health

Health endpoints and Kubernetes probes provide application health information.

The observability architecture can evolve toward:



Observability Stack



Dashboards

20. Security

Security is treated as a platform concern rather than only an application concern.

Key security areas include:

Container Security

- Minimal runtime images
- Non-root containers
- Image scanning
- Dependency scanning
- Controlled image versions

Kubernetes Security

- RBAC
- Service Accounts
- Namespace isolation
- Network Policies
- Pod Security
- Secret management

Application Security

- Authentication
- Authorization
- Input validation
- Secure API design
- Dependency management

Infrastructure Security

- Least privilege IAM
 - Network segmentation
 - Private workloads
 - Encryption
 - Secure secrets management
-

21. Local Development Environment

The platform can be developed locally using:

- Windows
- Docker Desktop
- Minikube
- kubectl
- Helm
- Maven
- Java 21
- Git

Verify Java:

```
java -version
```

Verify Maven:

```
mvn -version
```

Verify Docker:

```
docker version
```

Verify Kubernetes:

```
kubectl version
```

Verify Helm:

```
helm version
```

Verify Minikube:

```
minikube version
```

22. Running the Platform

Start Minikube:

```
minikube start --driver=docker
```

Verify the cluster:

```
kubectl get nodes
```

Expected result:

NAME	STATUS	ROLES	AGE
minikube	Ready	control-plane	...

Create or verify the namespace:

```
kubectl get namespaces
```

23. Building the Applications

Navigate to an application:

```
cd applications\customer-service
```

Build the application:

```
mvn clean package
```

Skip tests when specifically required:

```
mvn clean package -DskipTests
```

The resulting JAR is normally located under:

```
target/
```

Tests should preferably remain enabled in CI.

24. Building Docker Images

Build an application image:

```
docker build -t customer-service:v3 .
```

Verify:

```
docker images
```

The image should appear in the local Docker image repository.

25. Deploying to Kubernetes

For local Minikube development, images can be made available to the cluster.

One approach is:

```
minikube image load customer-service:v3
```

Verify the image:

```
minikube image ls
```

Kubernetes can then use the locally available image.

26. Helm Deployment

List Helm releases:

```
helm list -A
```

Install a chart:

```
helm install customer-service ./helm/customer-service  
-n baraza-banking  
--create-namespace
```

Upgrade an existing release:

```
helm upgrade customer-service ./helm/customer-service  
-n baraza-banking
```

Inspect the release:

```
helm status customer-service -n baraza-banking
```

Render the manifests without deploying:

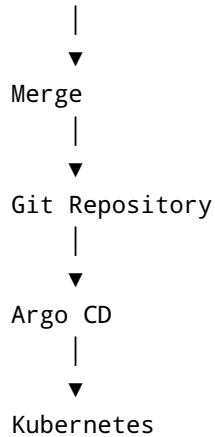
```
helm template customer-service ./helm/customer-service  
-n baraza-banking
```

This is particularly useful for debugging Helm templates.

27. GitOps Deployment

In a GitOps environment:

```
graph TD
  Developer --> GitCommit[Git Commit]
  GitCommit --> PullRequest[Pull Request]
  PullRequest --> Review
```



Argo CD becomes responsible for applying the desired state.

The cluster should therefore not be manually modified as the normal production deployment mechanism.

Manual `kubectl` changes may cause configuration drift.

28. Verification

After deployment:

```
kubectl get pods -n baraza-banking
```

Check deployments:

```
kubectl get deployments -n baraza-banking
```

Check services:

```
kubectl get services -n baraza-banking
```

Check ingress:

```
kubectl get ingress -n baraza-banking
```

Check HPA:

```
kubectl get hpa -n baraza-banking
```

Check detailed Pod information:

```
kubectl describe pod <pod-name> -n baraza-banking
```

Check logs:

```
kubectl logs <pod-name> -n baraza-banking
```

29. Troubleshooting

29.1 ImagePullBackOff

Check:

```
kubectl get pods -n baraza-banking
```

Then:

```
kubectl describe pod <pod-name> -n baraza-banking
```

Look at the Events section.

Common causes include:

- Image does not exist
- Incorrect image tag
- Image registry authentication failure
- Incorrect image name
- Kubernetes cannot access the registry
- Local Minikube image not loaded

29.2 CreateContainerConfigError

Inspect the Pod:

```
kubectl describe pod <pod-name> -n baraza-banking
```

Common causes include:

- Missing Secret
- Missing ConfigMap
- Incorrect environment variable reference
- Invalid configuration
- Missing required Kubernetes resource

29.3 CrashLoopBackOff

Check logs:

```
kubectl logs <pod-name> -n baraza-banking
```

For the previous crashed container:

```
kubectl logs <pod-name> -n baraza-banking --previous
```

Inspect:

```
kubectl describe pod <pod-name> -n baraza-banking
```

29.4 Service Not Reachable

Check:

```
kubectl get svc -n baraza-banking
```

Then verify endpoints:

```
kubectl get endpoints -n baraza-banking
```

Verify Pod labels:

```
kubectl get pods -n baraza-banking --show-labels
```

A Service selector must correctly match the labels on the Pods.

29.5 HPA Not Scaling

Check:

```
kubectl get hpa -n baraza-banking
```

Check metrics:

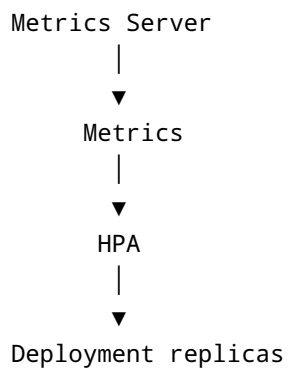
```
kubectl top pods -n baraza-banking
```

Check:

```
kubectl top nodes
```

If metrics are unavailable, investigate Metrics Server.

The relationship is:



29.6 PostgreSQL Connectivity Problems

Check PostgreSQL Pods:

```
kubectl get pods -n baraza-banking
```

Check logs:

```
kubectl logs <postgres-pod> -n baraza-banking
```

Check Service:

```
kubectl get svc -n baraza-banking
```

Check configuration and Secrets:

```
kubectl get secrets -n baraza-banking
```

Never expose database credentials in logs or source control.

29.7 Port Forwarding

For local database access:

```
kubectl port-forward svc/postgres 5432:5432 -n baraza-banking
```

The local machine can then connect to:

```
localhost:5432
```

If port forwarding fails, verify:

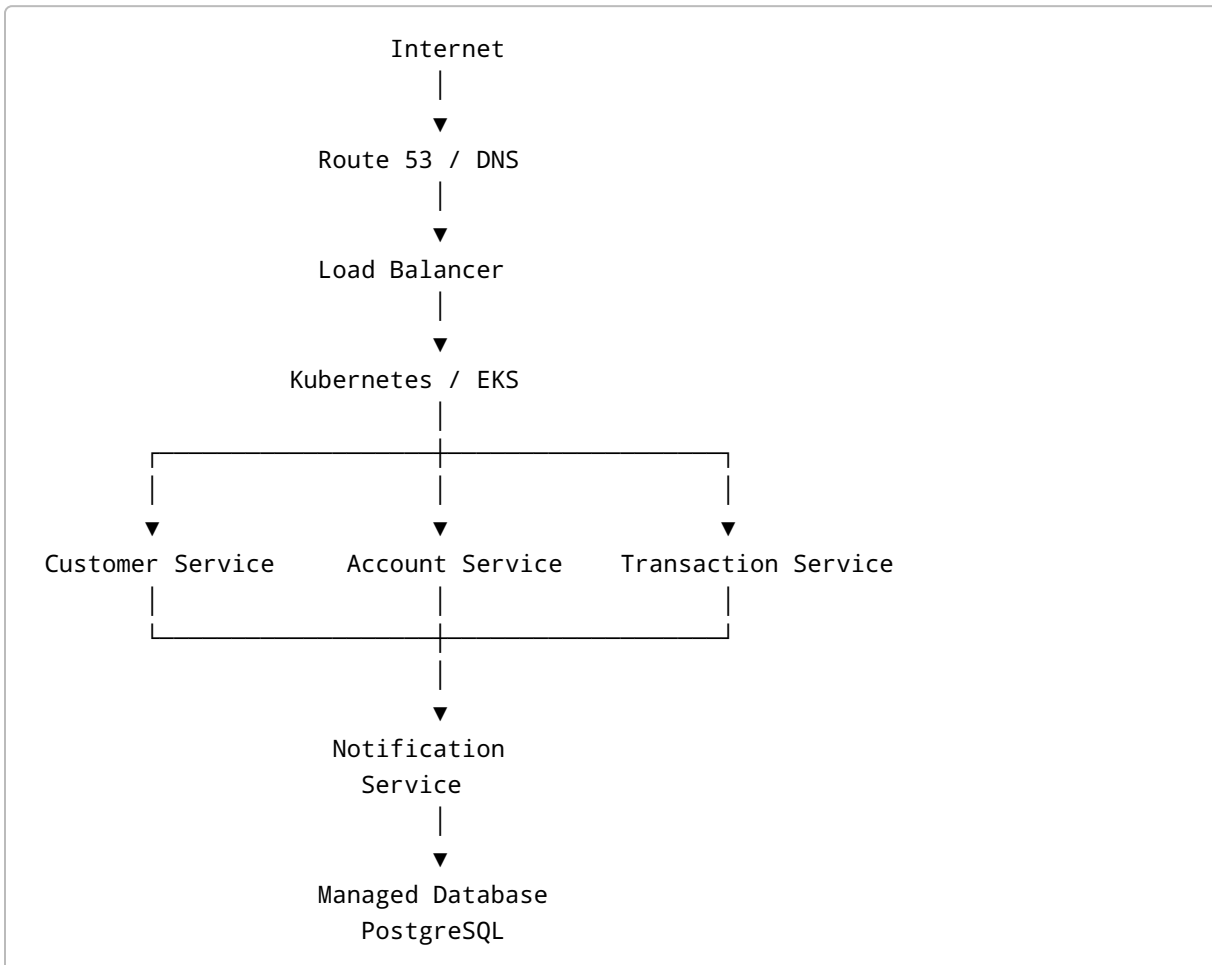
```
kubectl get pods -n baraza-banking  
kubectl get svc -n baraza-banking  
kubectl get endpoints -n baraza-banking
```

Also verify that the Kubernetes API server and Minikube cluster are healthy.

30. Production Architecture

The long-term production architecture is intended to evolve toward a cloud environment.

A conceptual AWS deployment could look like:



Supporting infrastructure would include:

- AWS
 - VPC
 - Public/Private Subnets
 - Security Groups
 - IAM
 - EKS
 - Load Balancer
 - PostgreSQL
 - Monitoring

31. Banking and Regulatory Considerations

Because Baraza represents a banking workload, production implementation must consider financial-sector requirements.

Areas include:

Data Protection

- Encryption at rest
- Encryption in transit
- Secure key management
- Data classification

Access Control

- Least privilege
- Strong authentication
- Role-based access
- Privileged access management

Auditability

Financial operations should produce immutable and traceable audit records.

Transaction Integrity

Transactions should support:

- Idempotency
- Atomicity
- Consistency
- Concurrency control
- Duplicate transaction protection
- Reliable transaction state management

Compliance

A production banking implementation may need to address applicable requirements such as:

- PCI DSS
- SOC controls

- Data protection regulations
- Banking regulations
- Internal security policies
- Audit requirements

Compliance requirements must be validated against the actual jurisdiction and deployment environment.

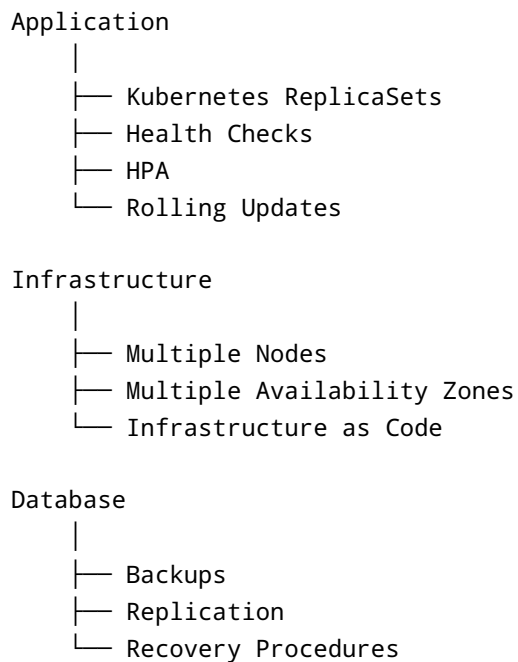
32. Disaster Recovery

A production banking platform must be designed for failure.

Potential failure scenarios include:

- Pod failure
- Node failure
- Availability Zone failure
- Database failure
- Application failure
- Network failure
- Deployment failure
- Configuration error

Recovery mechanisms include:

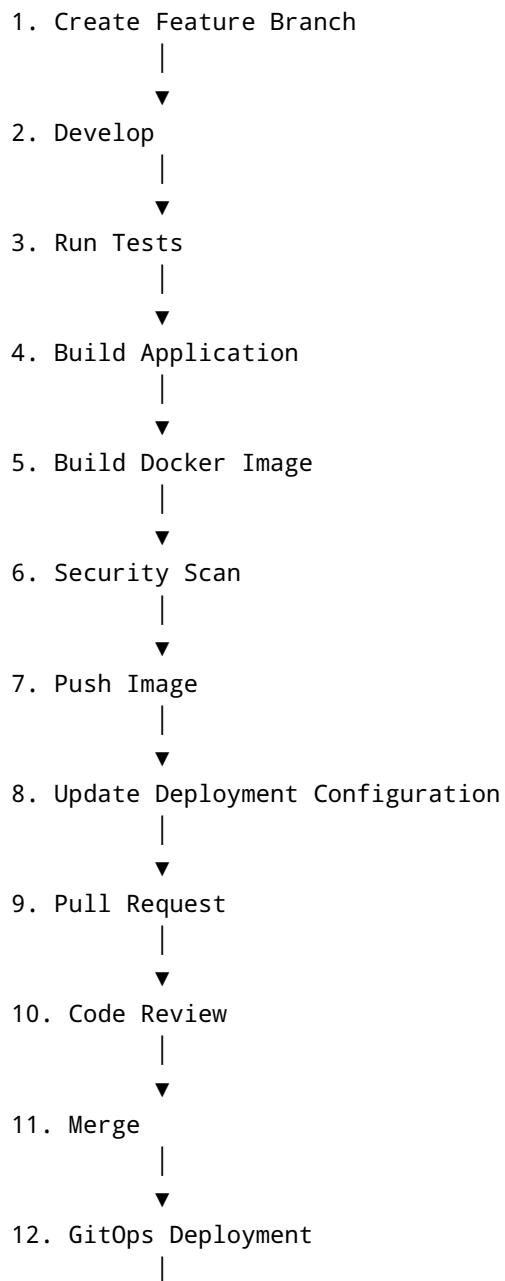


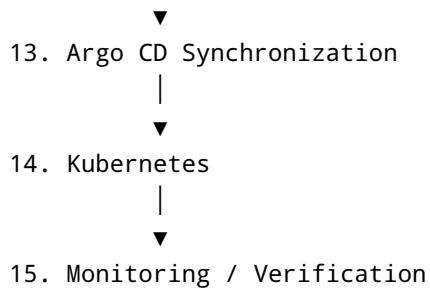
Recovery objectives should eventually be defined using:

- RTO — Recovery Time Objective
- RPO — Recovery Point Objective

33. Development Workflow

The preferred development workflow is:





34. Platform Engineering Principles

Baraza is designed around several Platform Engineering principles.

Infrastructure as Code

Infrastructure should be represented as version-controlled code.

Declarative Configuration

The desired state should be explicitly defined.

Automation

Manual repetitive operations should gradually be replaced with automation.

Self-Service

Development teams should eventually be able to deploy services through standardized platform interfaces.

Observability

Systems should provide sufficient visibility to understand their behavior.

Reliability

Failure should be expected and systems should recover automatically where possible.

Security by Design

Security controls should be incorporated into the development and deployment lifecycle.

Reproducibility

A deployment should be reproducible from version-controlled source and infrastructure definitions.

35. Current Project Status

This section is intentionally maintained as a living document.

Application Layer

- ☒ Spring Boot foundation
- ☒ Customer Service
- ☒ Account Service
- ☐ Transaction Service — continued development
- ☐ Notification Service — continued development
- ☐ Comprehensive integration testing

Containerization

- ☒ Dockerfiles
- ☒ Java runtime containers
- ☒ Non-root container configuration
- ☒ Docker image builds
- ☐ Automated image vulnerability scanning
- ☐ Production container registry strategy

Kubernetes

- ☒ Kubernetes cluster
- ☒ Namespace
- ☒ Deployments
- ☒ Services
- ☒ Configuration
- ☒ Health probes
- ☒ Resource requests/limits
- ☒ HPA
- ☐ Production-grade network policies
- ☐ Production-grade RBAC

Helm

- ☒ Helm chart structure
- ☒ Values configuration
- ☒ Environment-specific values
- ☒ Deployment templating
- ☒ Service templating

- ☐ Complete charts for all services
- ☐ Chart testing

Infrastructure

- ☒ Terraform foundation
- ☐ Complete cloud infrastructure
- ☐ Production Kubernetes infrastructure
- ☐ Managed database
- ☐ Production networking

GitOps

- ☒ GitOps architecture defined
- ☐ Complete Argo CD application definitions
- ☐ Automated synchronization
- ☐ Production GitOps structure

Observability

- ☒ Kubernetes metrics foundation
 - ☒ HPA metrics
 - ☐ Complete Prometheus implementation
 - ☐ Grafana dashboards
 - ☐ Centralized logging
 - ☐ Distributed tracing
-

36. Roadmap

Phase 1 — Application Platform

- ☒ Customer Service
- ☒ Account Service
- ☐ Transaction Service
- ☐ Notification Service
- ☐ API integration
- ☐ Automated testing

Phase 2 — Kubernetes Platform

- ☒ Deployments
- ☒ Services
- ☒ Configuration
- ☒ Probes
- ☒ Resource management
- ☒ HPA

- ☐ Network Policies
- ☐ RBAC hardening

Phase 3 — Helm

- ☒ Initial chart
- ☒ Values files
- ☒ Environment configuration
- ☐ Standardized charts
- ☐ Chart validation

Phase 4 — CI/CD

- ☐ Automated Maven builds
- ☐ Automated testing
- ☐ Docker image builds
- ☐ Image scanning
- ☐ Registry publishing
- ☐ Deployment automation

Phase 5 — GitOps

- ☐ Argo CD applications
- ☐ Automated synchronization
- ☐ Environment promotion
- ☐ Drift detection
- ☐ Rollback workflows

Phase 6 — Infrastructure

- ☐ Terraform AWS infrastructure
- ☐ VPC
- ☐ EKS
- ☐ Managed PostgreSQL
- ☐ IAM
- ☐ Load balancing
- ☐ Secrets management

Phase 7 — Observability

- ☐ Prometheus
- ☐ Grafana
- ☐ Centralized logging
- ☐ Alerting
- ☐ Distributed tracing
- ☐ SLO/SLI definitions

Phase 8 — Production Hardening

- ☐ Security scanning
 - ☐ Network policies
 - ☐ Pod security
 - ☐ Backup and recovery
 - ☐ Disaster recovery
 - ☐ High availability
 - ☐ Compliance controls
 - ☐ Performance testing
-

37. Future Improvements

Future iterations of Baraza may include:

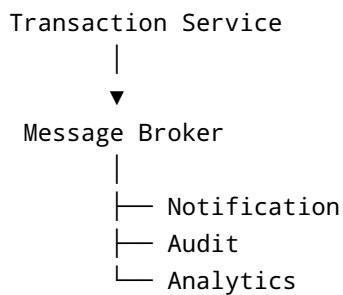
API Gateway

Introduce a dedicated API gateway for:

- Authentication
- Routing
- Rate limiting
- API policies
- Request transformation

Event-Driven Architecture

Introduce messaging infrastructure such as:



Service Observability

Add:

- OpenTelemetry
- Distributed tracing

- Application metrics
- Correlation IDs

Security

Introduce:

- External secrets management
- Container image signing
- Software supply-chain security
- Policy as Code
- Runtime security

Platform Automation

Introduce reusable platform components for:

- New services
- New environments
- Standard deployments
- Standard monitoring
- Standard security policies

38. Documentation

Detailed documentation will be maintained under:

```
docs/
```

Planned documentation includes:

```
docs/  
|  
├─ architecture.md  
├─ platform-overview.md  
├─ kubernetes.md  
├─ helm.md  
├─ docker.md  
├─ terraform.md  
├─ gitops-argocd.md  
├─ ci-cd.md  
├─ monitoring.md  
└─ security.md
```

```
|— networking.md
|— database.md
|— troubleshooting.md
|
└─ runbooks/
    |— deployment.md
    |— rollback.md
    |— image-management.md
    └─ kubernetes-debugging.md
```

The README serves as the high-level entry point while the `docs/` directory contains detailed technical procedures.

39. Contributing

Development should follow the standard Git workflow.

```
Feature Branch
  |
  ▼
Development
  |
  ▼
Tests
  |
  ▼
Pull Request
  |
  ▼
Code Review
  |
  ▼
Merge
```

Before submitting changes:

```
mvn clean test
```

Application-specific validation should also be performed.

Infrastructure changes should be reviewed before applying them to shared environments.

40. License

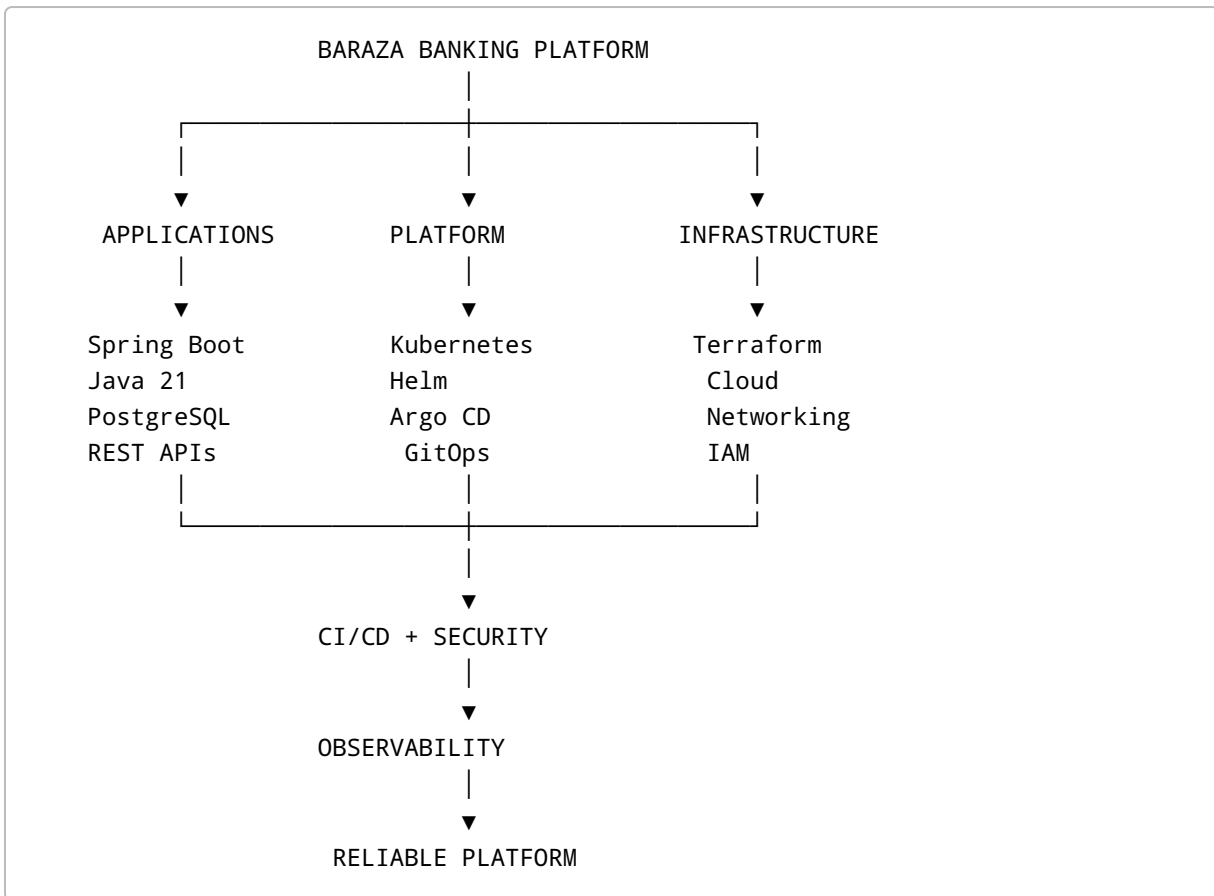
This project is currently intended as a development, learning, and portfolio platform.

The licensing model should be defined before the platform is distributed publicly or used commercially.

Project Vision

Baraza Banking Platform is intended to evolve beyond a simple Spring Boot application.

The long-term objective is to demonstrate a complete modern banking technology platform:



The project will continuously evolve toward a production-grade cloud-native banking platform demonstrating:

Software Engineering + DevOps + Platform Engineering + Cloud Engineering + Security + Observability + GitOps.